

国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第八章：PID控制器设计方法

Chapter 8 Design Methods for PID Controllers



Professor Dingyu Xue, xuedingyu@mail.neu.edu.cn
School of Information Science and Engineering,
Northeastern University, Shenyang, CHINA



本章主要内容

- PID控制器简介
- PID控制器的经典设计方法
- 最优PID控制器设计界面
 - OptimPID界面简介
 - PID控制器设计举例
 - 开放的界面设计框架

国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第八章 PID控制器设计方法

PID控制器简介

An Introduction to PID Controllers

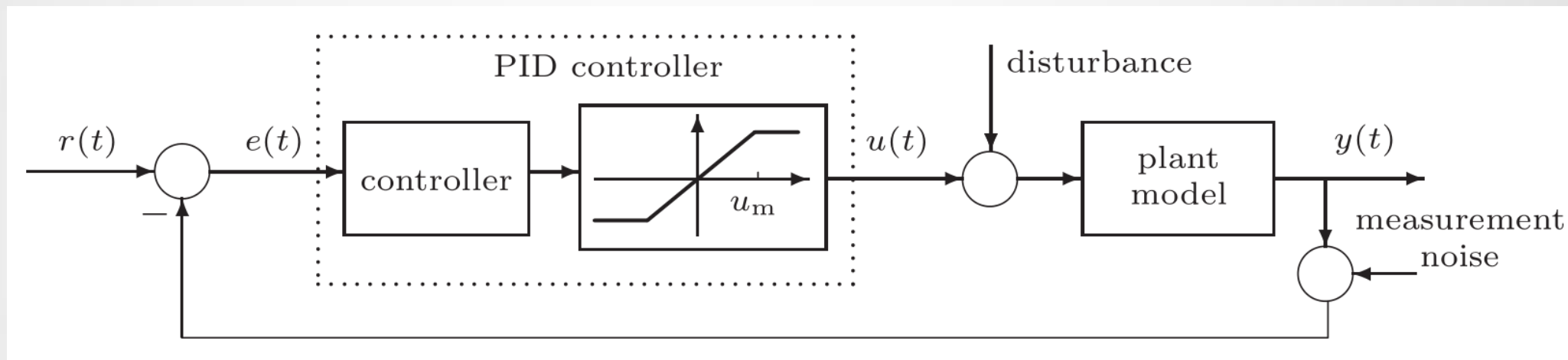


主讲：薛定宇教授



PID控制器设计概述

➤ PID控制结构



➤ 本节主要内容

- 连续PID控制器与离散PID控制器
- PID控制器的变形



连续**PID**控制器的数学模型

➤ PID控制器的一般形式

$$u(t) = K_p e(t) + K_i \int_0^t e(\tau) d\tau + K_d \frac{de(t)}{dt}$$

➤ 两种基本表示模型

➤ 标准型PID控制器

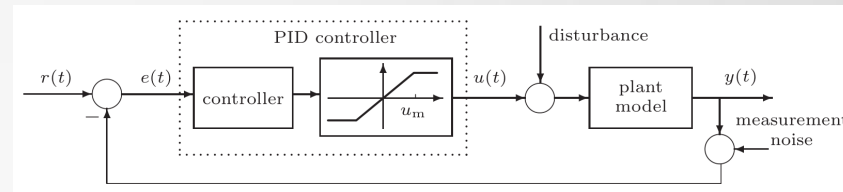
$$G_c(s) = K_p \left(1 + \frac{1}{T_i s} + \frac{T_d s}{T_d / N s + 1} \right)$$

$$G_c = \text{pidstd}(K_p, T_i, T_d, N)$$

➤ 并联型PID控制器

$$G_c(s) = K_p + \frac{K_i}{s} + \frac{K_d s}{T_f s + 1}$$

$$G_c = \text{pid}(K_p, K_i, K_d, T_f)$$





离散**PID**控制器

➤ 微积分的离散近似 $\frac{de(t)}{dt} \simeq \frac{e(kT) - e[(k-1)T]}{T}$

$$\int_0^{kT} e(t)dt \simeq T \sum_{i=0}^k e(iT) = \int_0^{(k-1)T} e(t)dt + Te(kT)$$

➤ 离散PID控制器的一般形式

$$u(kT) = K_p e(kT) + K_i T \sum_{m=0}^k e(mT) + \frac{K_d}{T} [e(kT) - e[(k-1)T]]$$

➤ 简记 $u_k = K_p e_k + K_i T \sum_{m=0}^k e_m + \frac{K_d}{T} (e_k - e_{k-1})$



两种不同的微分近似及**PID**控制器

➤ 后向Euler微分

$$u_k = K_p e_k + K_i T \sum_{m=0}^k e_m + \frac{K_d}{T} (e_k - e_{k-1})$$

$$G_c(z) = K_p + \frac{K_i T z}{z - 1} + \frac{K_d (z - 1)}{T z}$$

➤ 前向Euler微分

$$u_k = K_p e_k + K_i T \sum_{m=0}^{k+1} e_m + \frac{K_d}{T} (e_{k+1} - e_k)$$

$$G_c(z) = K_p + \frac{K_i T}{z - 1} + \frac{K_d (z - 1)}{T}$$



例8-1 不同PID控制器输入

➤ 不同的控制器 $T = 0.1s$

$$C_1(s) = 1.5 + \frac{5.2}{s} + 3.5s, \quad C_2(s) = 1.5 \left(1 + \frac{3.5s}{1 + 0.035s} \right)$$

$$C_3(z) = 1.5 + \frac{5.2}{z-1} + 3.5(z-1), \quad C_4(z) = 1.5 \left(1 + \frac{z}{5.2(z-1)} + \frac{3.5(z-1)}{z} \right)$$

➤ MATLAB输入语句



```
>> C1=pid(1.5,5.2,3.5,0), C2=pidstd(1.5,inf,3.5,100)  
C3=pid(1.5,52,0.35,0,0.1)  
C4=pidstd(1.5,52,0.35,inf,0.1,'IFormula','backward')
```




变形的PID控制器

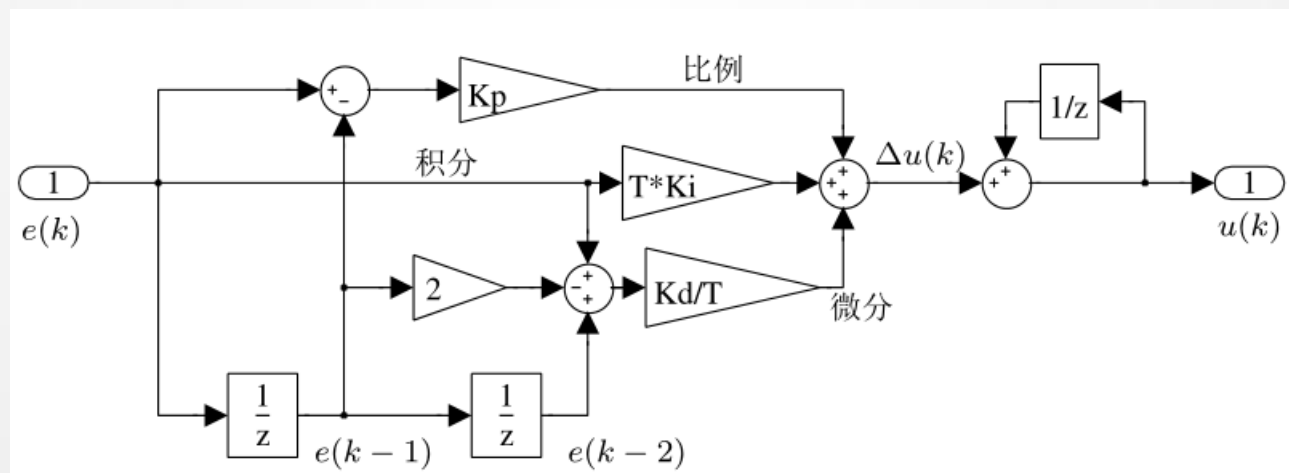
- 积分分离式PID控制器
 - 误差大时关闭 I , 快速跟踪
 - 误差小时开启 I , 消除静态误差
- 离散增量式PID控制器
 - 误差的积分累加计算麻烦
 - 可以引入增量来计算积分量
 - 控制量的增量

$$u_k - u_{k-1} = K_p(e_k - e_{k-1}) + K_i T e_k + \frac{K_d}{T}(e_k + e_{k-2} - 2e_{k-1})$$



增量式PID控制器

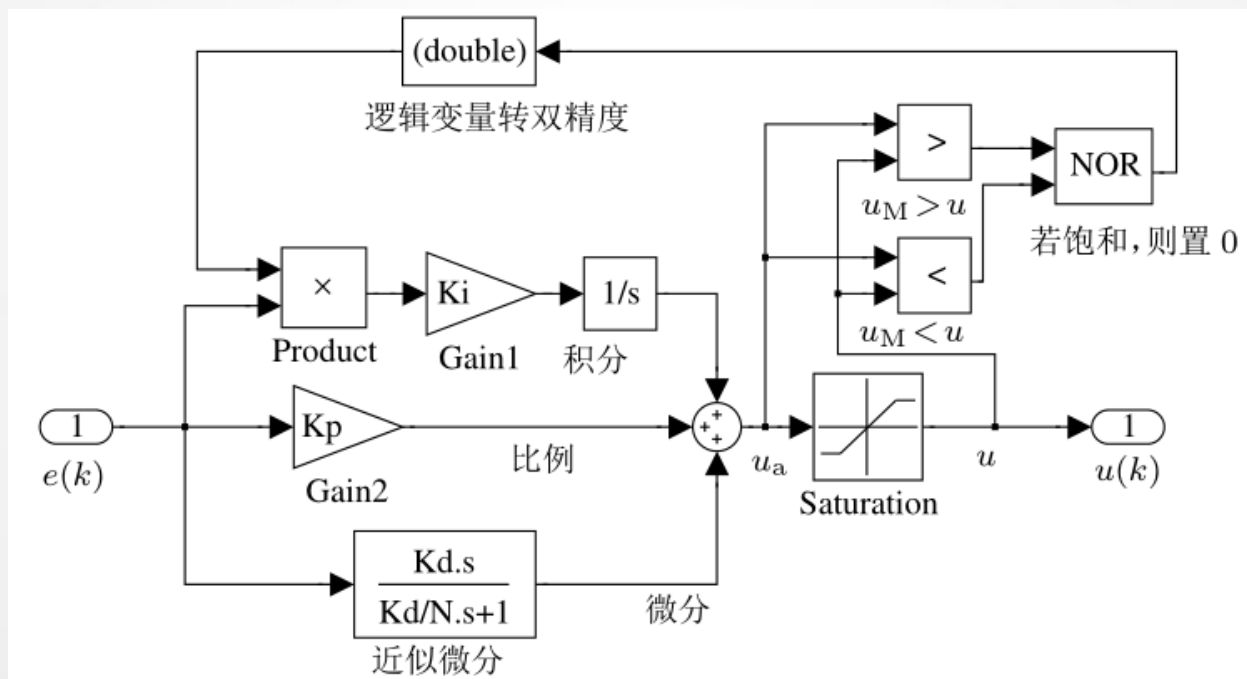
- 控制量计算 $u_k = u_{k-1} + \Delta u_k$
- 增量式PID控制器Simulink框图
- 文件名：c8mdpid1.mdl





抗积分饱和和PID控制器

- 抗积分饱和 (anti-windup)
- PID控制器结构：c8mantiw.mdl





PID控制器小结

- 两种不同的连续PID控制器
 - 直接输入方法：pid, pidstd
- 离散PID控制器
 - 积分算法
 - 特殊结构



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第八章 PID控制器设计方法

PID设计的经典方法

Classical Design of PID Controllers



主讲：薛定宇教授



PID控制器设计经典方法

- 常用过程对象一阶延迟模型(FOPDT)的获取
- 基于FOPDT的设计方法
 - Ziegler-Nichols经验公式
 - MATLAB实现
- 其他模型类型的直接设计
 - MATLAB直接求解
 - 存在的问题——设计方法；非线性



过程对象的一阶延迟近似

➤ 常见近似模型 FOPDT $G(s) = \frac{k}{Ts + 1} e^{-Ls}$

➤ 问题，如何求出 k, L, T

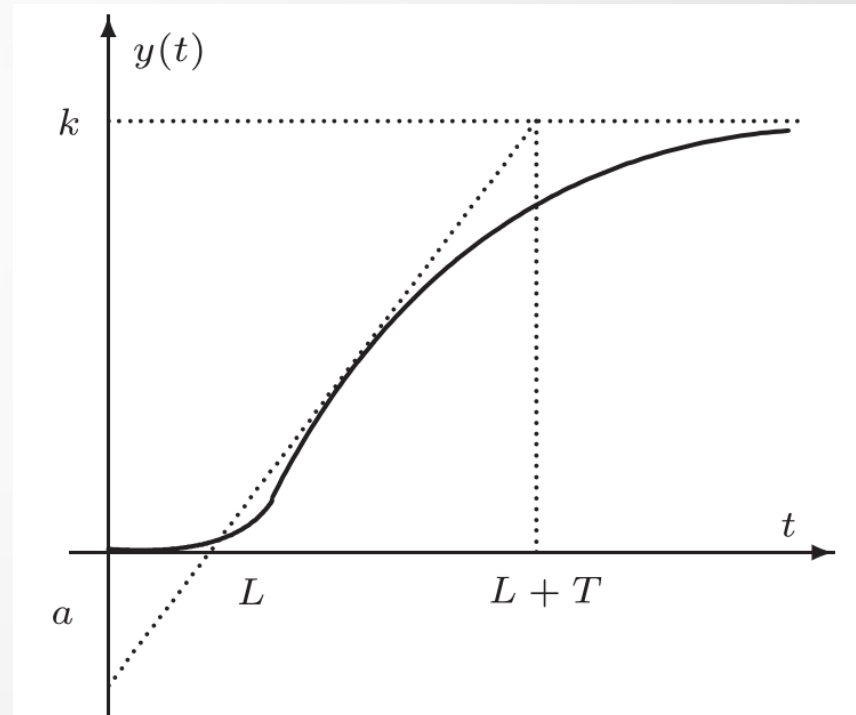
➤ 读图方法

➤ 最优降阶方法——0/1

➤ 最小二乘参数识别

➤ 基于MATLAB的求解

$[K, L, T, G_c] = \text{getfopdt}(\text{key}, G)$





例8-2 参数识别

➤ 受控对象 $G(s) = \frac{1}{(s+1)^5}$

➤ 获取FOPDT模型的方法



```
>> s=tf('s'); G=1/(s+1)^5;  
[K1,L1,T1,G1]=getfopdt(1,G),  
[K2,L2,T2,G2]=getfopdt(2,G)  
[K3,L3,T3,G3]=getfopdt(3,G)  
[K4,L4,T4,G4]=getfopdt(4,G)  
step(G,'-',G1,':',G2,'*',G3,'--',G4,'-.',15)
```



Ziegler-Nichols经验公式

- 1942年Ziegler与Nichols的一篇经典文章
- 已知： k, L, T 参数 $a = KL/T$
- 求解PID类控制器参数
- 查表方法（直接计算、编MATLAB程序）

控制器类型	由阶跃响应整定			由频域响应整定		
	K_p	T_i	T_d	K_p	T_i	T_d
P	$1/a$			$0.5K_c$		
PI	$0.9/a$	$3L$		$0.4K_c$	$0.8T_c$	
PID	$1.2/a$	$2L$	$L/2$	$0.6K_c$	$0.5T_c$	$0.12T_c$



基于MATLAB的设计程序

➤ 直接设计

$$[G_c, K_p, T_i, T_d] = \text{ziegler}(\text{key}, \text{vars})$$

➤ 时域公式直接设计 $\text{vars} = [K, L, T, N]$

➤ 频域公式直接设计

$$\text{vars} = [K_c, T_c, N], \quad K_c, T_c = 2\pi/\omega_c$$

➤ 改进方法 —— 可调参数 r_b, ϕ_b

$$[G_c, K_p, T_i, T_d] = \text{ziegler}(\text{key}, [K_c, T_c, r_b, \phi_b, N])$$



例8-3 直接设计

K_p	T_i	T_d
$1/a$		
$0.9/a$	$3L$	
$1.2/a$	$2L$	$L/2$

➤ 受控对象 $G(s) = \frac{0.1}{(s+1)^6}$

➤ 直接设计 $k = 0.1, T = 2.883, L = 3.37$

➤ 控制器设计与效果比较



```
>> s=tf('s'); G=0.1/(s+1)^6; N=10;  
K=0.1; T=2.883; L=3.37; a=K*L/T;  
Kp=0.9/a; Ti=3*L; G1=Kp*(1+tf(1,[Ti 0]));  
Kp=1.2/a; Ti=2*L; Td=0.5*L; p=[Kp,Ti,Td]
```



```
>> G2=Kp*(1+tf(1,[Ti,0])+tf([Td 0],[Td/N 1]));  
step(feedback(G*G1,1),'-',feedback(G*G2,1),'--')
```



不同设计方法设计的控制器

➤ 频率响应数据设计



```
>> [Kc,b,wc,d]=margin(G); Tc=2*pi/wc;  
Kp=0.4*Kc; Ti=0.8*Tc; G1=Kp*(1+tf(1,[Ti 0]));  
Kp=0.6*Kc; Ti=0.5*Tc; Td=0.12*Tc;  
G2=Kp*(1+tf(1,[Ti,0])+tf([Td 0],1));  
step(feedback(G*G1,1),'-',feedback(G*G2,1),'--')
```

K_p	T_i	T_d
$0.5K_c$		
$0.4K_c$	$0.8T_c$	
$0.6K_c$	$0.5T_c$	$0.12T_c$

➤ 改进设计方法



```
>> s=tf('s'); G=0.1/(s+1)^6;  
G1=pidtune(G,'pi'), G2=pidtune(G,'pid')  
step(feedback(G*G1,1),feedback(G*G2,1))
```



其他基于模型的设计方法

➤ PI/PID设计手册

➤ Aidan O'Dwyer. Handbook of PI and PID controller tuning rules, Imperial College Press, 2001, 2006, 2009

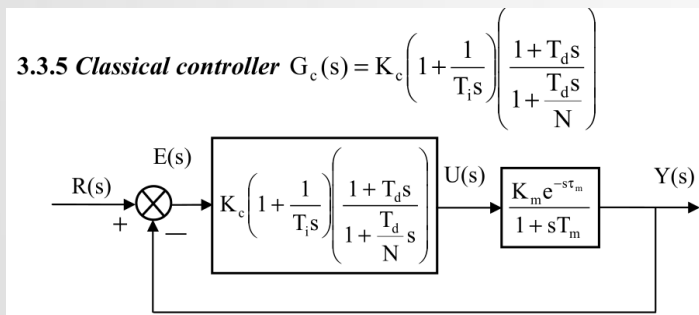
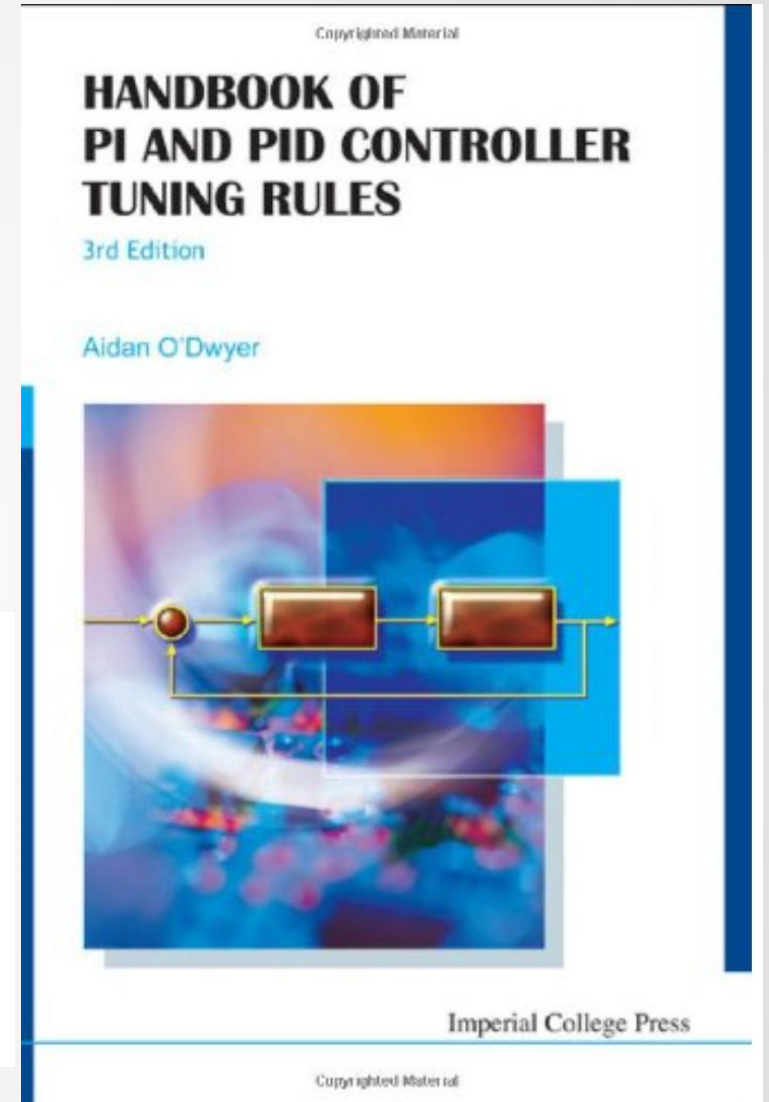


Table 13: PID controller tuning rules – FOLPD model $G_m(s) = \frac{K_m e^{-s\tau_m}}{1 + sT_m}$

Rule	K_c	T_i	T_d	Comment
Process reaction				
Kraus (1986). Model: Method 23	$0.833T_m/K_m\tau_m$	$1.5\tau_m$	$0.25\tau_m$	$N = \infty$
	Foxboro EXACT controller pre-tuning. Also given by Hang <i>et al.</i> (1993b), p. 76.			
Witt and Waggoner (1990).	$x_1 T_m / K_m \tau_m$	τ_m	τ_m	$10 \leq N \leq 20$; $0.6 \leq x_1 \leq 1.0$
	Model: Method 2. 'Equivalent to' Ziegler and Nichols (1942).			
Witt and Waggoner (1990).	$^1 K_c$	T_i	T_d	$10 \leq N \leq 20$
	Model: Method 2. 'Preferred' tuning.			





MATLAB提供的设计函数

- Ziegler-Nichols 类方法的局限性
 - 只能用于 FOPDT 受控对象模型
 - 需要任意受控对象的PID控制器设计程序
- MATLAB的 pidtune 函数可以用于任意SISO LTI对象
 - 选项 —— p, i, pi, pd, pdf, pid, pidf

$$G_c = \text{pidtune}(G, 'pi', \omega_c), \quad G_c = \text{pidtune}(G, 'pid', \omega_c)$$



```
>> s=tf('s'); G=0.1/(s+1)^6;  
G1=pidtune(G,'pi'), G2=pidtune(G,'pidf')  
step(feedback(G*G1,1),feedback(G*G2,1))
```



例8-4 复杂受控对象的

➤ 受控对象

$$G(s) = \frac{1 + \frac{3e^{-s}}{s+1}}{s+1}$$

➤ 设计命令

➤ 用LTI模型表示复杂受控对象模型，直接设计控制器



```
>> s=tf('s'); G1=3/(s+1); G1.ioDelay=1;  
G=(1+G1)/(s+1);  
Gc1=pidtune(G,'pi'), Gc2=pidtune(G,'pidf')  
step(feedback(Gc1*G,1),feedback(Gc2*G,1))
```



不稳定模型的**PID**控制器设计

➤ 不稳定受控对象 $G(s) = \frac{s+2}{s^4 + 8s^3 + 4s^2 - s + 0.4}$,

➤ PID控制器

```
 >> s=tf('s'); G=(s+2)/(s^4+8*s^3+4*s^2-s+0.4);  
      Gc=pidtune(G,'pidf'), step(feedback(Gc*G,1))
```

```
 >> Gc=pidtune(G,'pidf',1), step(feedback(Gc*G,1))
```

➤ 真需要这么大的调节时间吗？

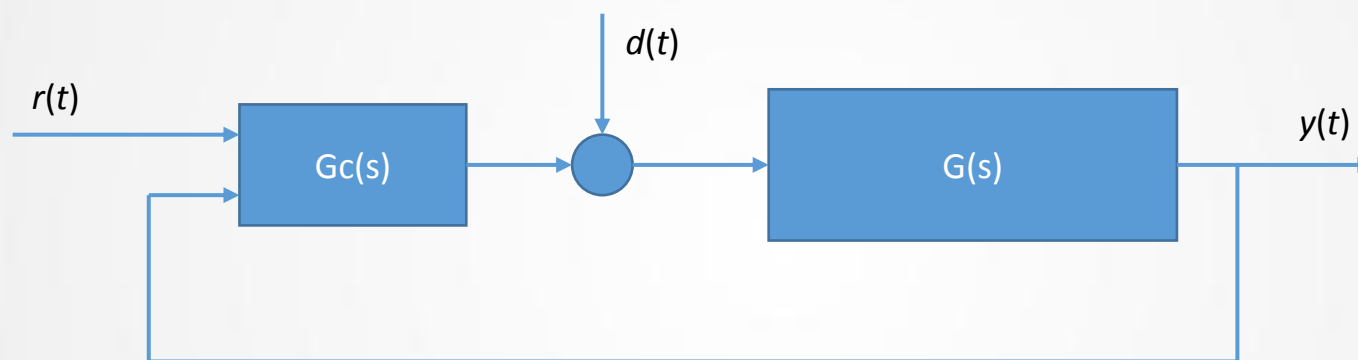
➤ 控制信号的曲线

```
 >> step(feedback(Gc,G))
```



例8-5 二自由度PID控制器

➤ 二自由度PID控制器结构



➤ 设计界面——pidTuner

$$G(s) = \frac{1 + \frac{3e^{-s}}{s+1}}{s+1}$$



```
>> s=tf('s'); G1=3/(s+1); G1.ioDelay=1;  
G=(1+G1)/(s+1); Gc1=pidTuner(G,'pidf2')
```



PID经典设计小结

- 针对一大类过程模型的近似设计方法
 - FOPDT模型的参数识别方法
 - 针对FOPDT模型的设计方法，Ziegler Nichols方法
 - 二自由度PID控制结构与设计
 - 存在的问题 —— 算法繁杂，不宜比较
- MATLAB提供的设计函数
 - 简单使用，效果良好
 - 不能处理时变模型、非线性模型等，效果有待改进



国家精品课程/ 国家精品资源共享课程/ 国家级精品教材

国家级十一(二)五规划教材/ 教育部自动化专业教学指导委员会牵头规划系列教材

控制系统仿真与CAD

第八章 PID控制器设计方法

最优PID控制器设计界面

Optimal PID Controller Design Interface



主讲：薛定宇教授



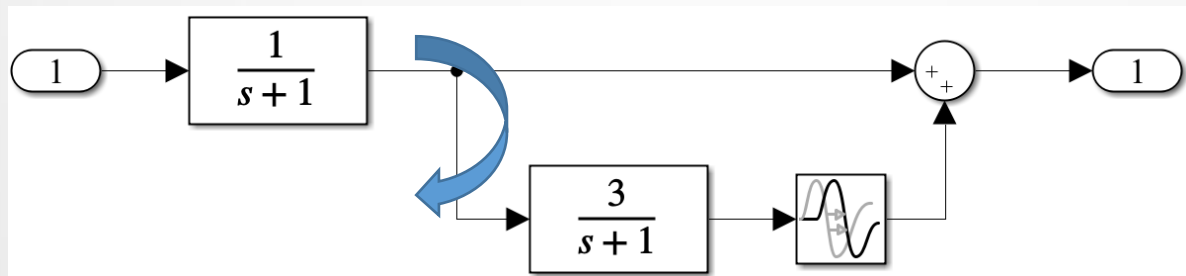
OptimPID: 最优PID控制器设计

- 为什么需要这样一个程序？
 - 现有程序很难处理复杂受控对象
 - 很难处理非线性
 - 没有切实可行的指标
- 我们编写了一个通用的PID控制器设计界面
 - 用Simulink描述受控对象（SISO，任意复杂）
 - 利用界面直接选择目标函数、控制器类型、最优优化方法、执行器饱和
 - 实现闭环系统的仿真与控制器再设计



受控对象模型的描述

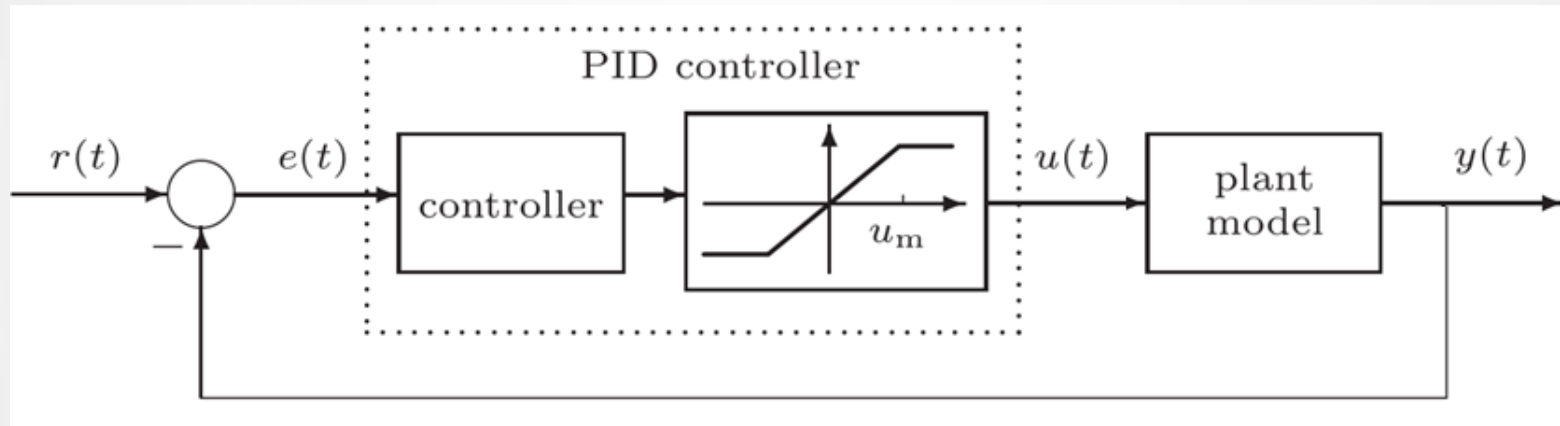
- 线性受控对象的预设模型 —— `mod_lti.mdl`
 - 需要提供 G , τ 两个变量
 - 只要是单变量LTI模型即可，连续、离散、带有内部延迟
- 非线性受控对象
 - 在输入端子与输出端子间的任意单变量Simulink模型





PID 控制系统的结构

➤ 典型问题 —— 闭环控制系统



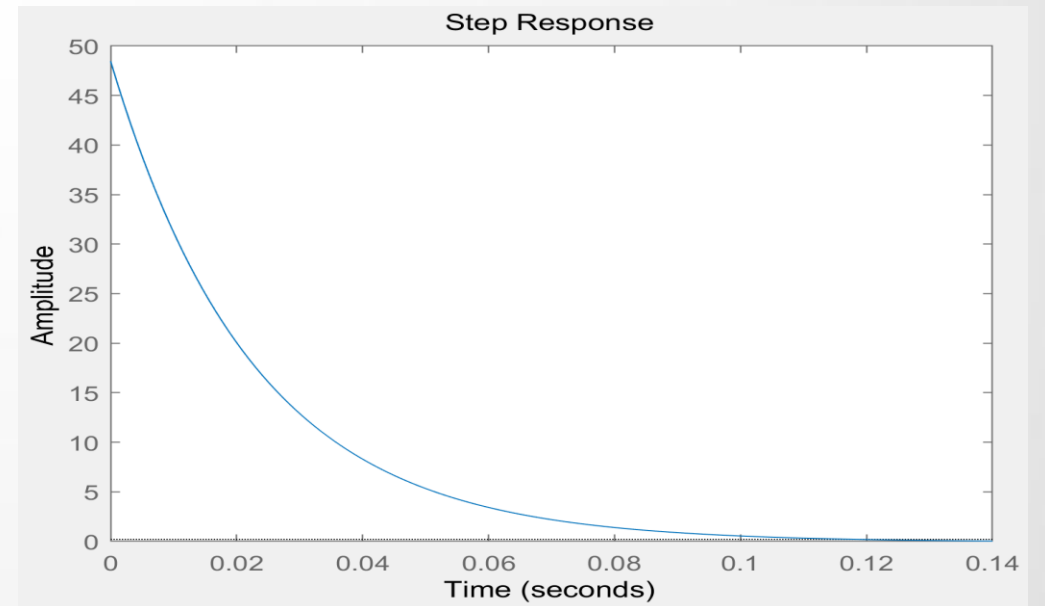
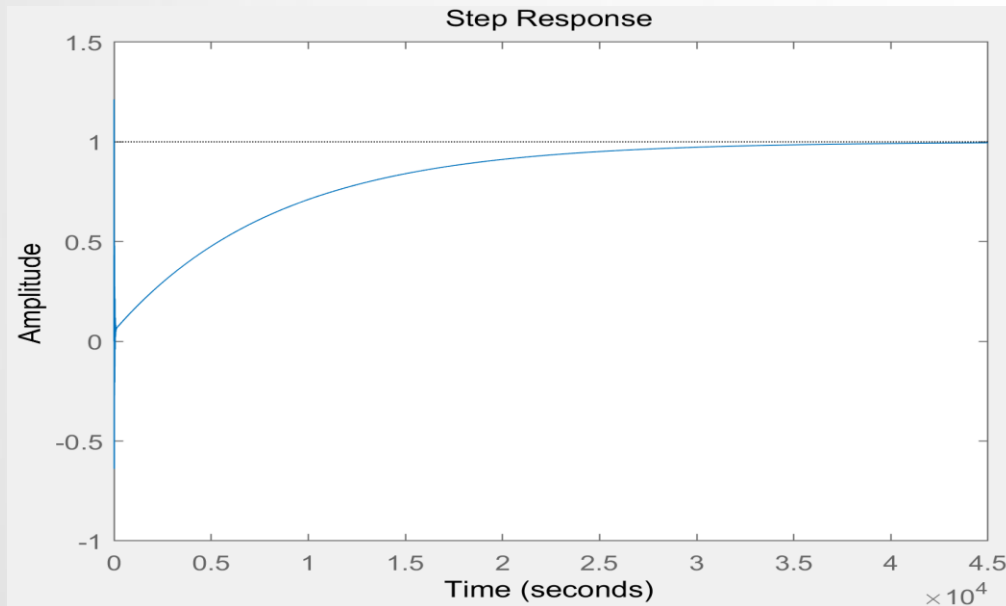
➤ PID类控制器

- 连续、离散；P，PI，PD，PID，抗积分饱和
- 可以带有饱和非线性



例8-6 不稳定受控对象的控制器设计

- 受控对象 $G(s) = \frac{s + 2}{s^4 + 8s^3 + 4s^2 - s + 0.4}$,
- 由pidtune直接设计的控制器





由OptimPID直接设计

➤ 线性模型参数输入



```
>> num=[1,2]; den=[1 8 4 -1 0.4];  
G=tf(num,den); tau=0;
```

➤ 控制器的驱动饱和 $|u(t)| < 5$

➤ 如何设计控制器

➤ 直接采用LTI模型的模板 mod_lti

➤ 选择终止仿真设计 $t_n = 10$

➤ 控制器类型、ITAE准则

The screenshot shows the 'PID Controller Optimizer' window with the following settings:

- Plant model name:** mod_1
- Terminate Time:** 10
- Controller Type:** PID Controller (selected from a list including P, PI, PD, and I controllers). The 'Continuous' radio button is selected.
- Actuator Saturation:** Lower Limit is -inf, Upper Limit is inf.
- Optimization Algorithm:** MATLAB Optimization (selected from a list including Genetic Algorithm, GAOT Toolbox, and Particle Swarm Optimization).
- Optimization Criterion:** ITAE Criterion (selected from a list including ISE and IAE criteria).
- Maximum overshoot at:** 3% (checkbox is unchecked).
- Controller Parameters:** Tuned Controller, Lower Bounds, and Upper Bounds fields are empty.
- Staircase Waveform:** t vector is [0], y vector is [1].
- Simulation Range:** 100 (checkbox is unchecked).
- Loop in optimization Tol=** 1e-3 (checkbox is checked).

Buttons on the right include: Create File, Refresh, Show Plant, Simulation, Show File, Tutorial, Optimize, and Exit.



例8-7 复杂 LTI 对象

➤ 复杂受控对象

$$G(s) = \frac{1 + \frac{3e^{-s}}{s+1}}{s+1}$$

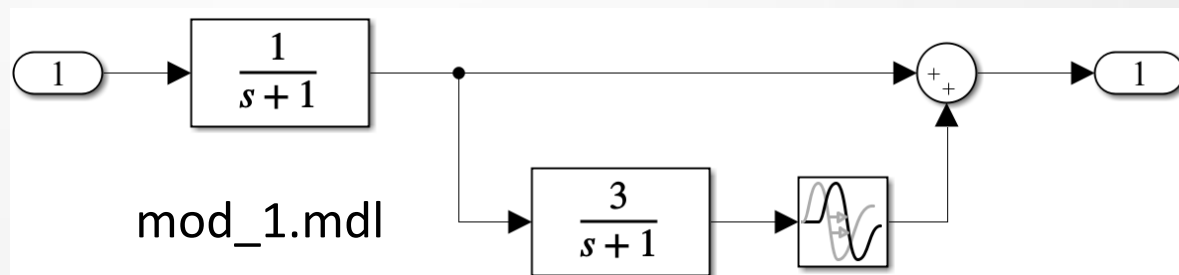
➤ OptimPID设计

➤ 线性受控对象使用 mod_lti.mdl



```
>> s=tf('s'); G1=3/(s+1); G1.ioDelay=1;  
G=(1+G1)/(s+1); tau=0;
```

➤ 非线性受控对象



➤ 选择大的终止时间：40



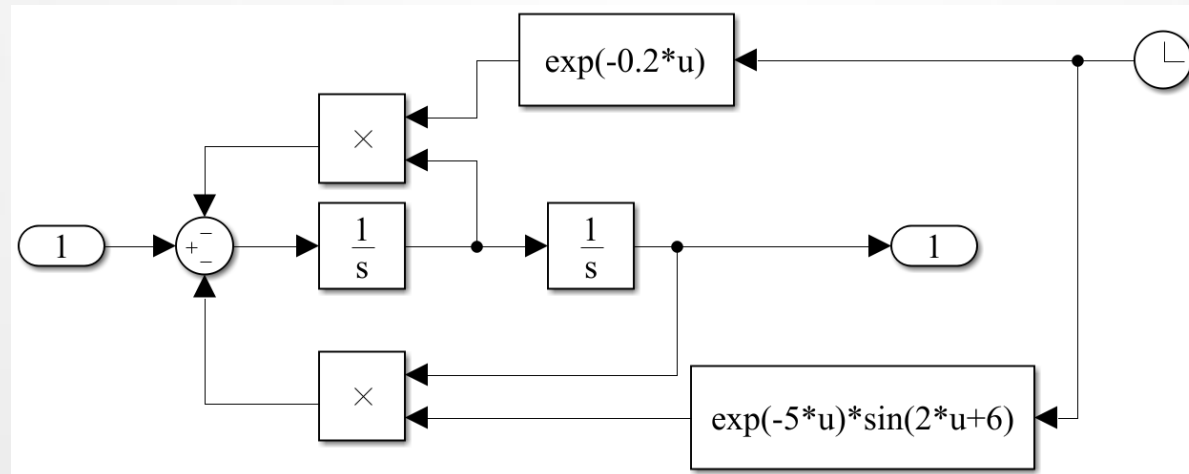
例8-15 时变系统控制

➤ 时变受控对象模型

$$\ddot{y}(t) + e^{-0.2t}\dot{y}(t) + e^{-5t}\sin(2t + 6)y(t) = u(t)$$

➤ 驱动饱和 , $|u(t)| < 2, t_n = 20$

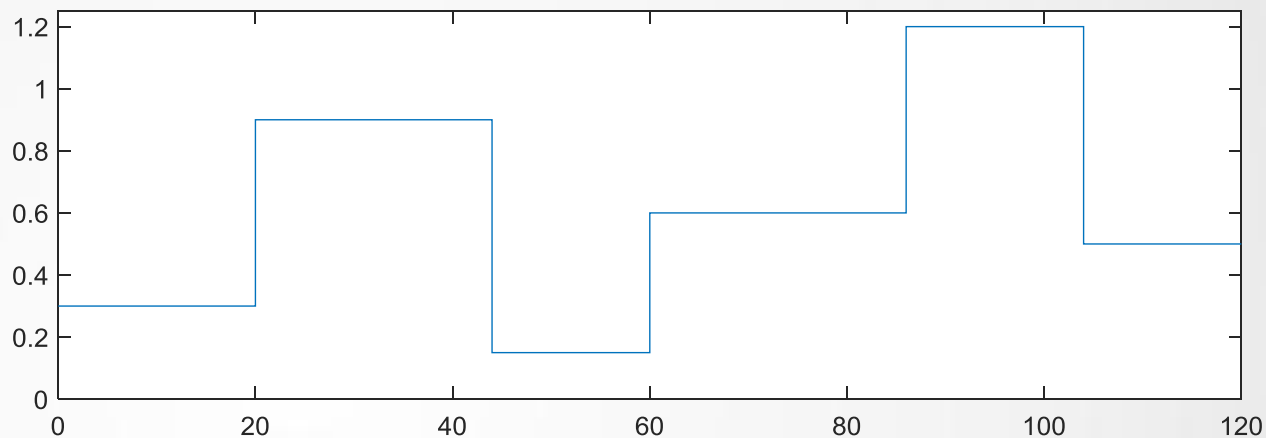
➤ 受控对象的Simulink建模 : mod_3.mdl





系统的阶梯信号响应

➤ 自定义阶梯信号



➤ 阶梯信号的输入



`[0,20,44,60,86,104]`



`[0.3,0.9,0.15,0.6,1.2,0.5]`



最优**PID**控制器设计小结

- 介绍了最优PID控制器设计的思路
- 给出了OptimPID程序的介绍，并通过例子演示了控制器设计与仿真方法
 - 一般线性系统
 - 驱动饱和的引入
 - 时变受控对象的控制与设计

